MyCodeSchool

Teorie — Spring Core Fundamentals

> Referință de adnotări pe care le întâlnești în prima săptămână de Spring. Citește o dată înainte de exerciții, întoarce-te la ea când lucrezi la ele.

1. Ce e Spring (în 3 propoziții)

Spring e un *container de obiecte*. Tu îi spui prin adnotări ce clase contează (bean-uri), el le instanțiază, le leagă între ele (dependency injection) și ți le dă când ai nevoie. Spring Boot e Spring + opinii implicite + auto-configurare, ca să nu mai scrii XML.

ApplicationContext = registrul tuturor bean-urilor. La pornire, Spring construiește contextul; după, codul tău cere bean-uri din el (sau le primește direct prin DI).

2. Bootstrap

`@SpringBootApplication`

Adnotare *combinată* — pune pe clasa cu `main()`. Înlocuiește trei adnotări:

componentă	rol
`@Configuration`	clasa poate defini `@Bean`-uri
`@EnableAutoConfiguration`	Spring Boot pornește auto-configurarea (DB, web, etc. în funcție de ce e pe classpath)
`@ComponentScan`	Spring scanează pachetul curent și sub-pachetele pentru `@Component` & co.

```
@SpringBootApplication
public class FundamentalsApplication {
    public static void main(String[] args) {
        SpringApplication.run(FundamentalsApplication.class, args);
    }
}
```

Reține: poziționarea clasei principale e importantă. `@ComponentScan` pornește din *pachetul ei* — clasele dintr-un pachet "vecin" nu sunt descoperite.

3. Definirea bean-urilor

3.1 Prin component scan (clase pe care le deții)

adnotare	rol semantic
`@Component`	generic — folosește când nu se potrivește o specializare
`@Service`	logică de business
`@Repository`	acces la date (DB, fișiere, API extern). În plus: traduce excepțiile de persistență la `DataAccessException`
`@Controller` / `@RestController`	endpoint-uri HTTP (Le vezi în L02)

Toate produc același efect tehnic: clasa devine bean. Distincția e *intenție*.

```
@Service
public class Calculator {
    public int add(int a, int b) { return a + b; }
}
```

3.2 Explicit prin `@Bean`

Când clasa nu e a ta (JDK, librării), nu poți pune `@Component` pe ea. Soluția: o definești într-o clasă `@Configuration`:

```
@Configuration
public class ClockConfig {

    @Bean
    public Clock clock() {
        return Clock.system(ZoneId.of("Europe/Bucharest"));
    }
}
```

- Numele bean-ului = numele metodei (`clock`).
- Spring apelează metoda *o singură dată* la pornire și păstrează obiectul în context (scope singleton — vezi §7).

4. Dependency Injection — cum cer bean-uri

4.1 Constructor injection (preferat ✅)

```
@Service
public class Alerts {
    private final NotificationSender sender;

    public Alerts(NotificationSender sender) {
        this.sender = sender;
    }
}
```

- **De ce e preferat:** dependențele sunt `final` (imutabile), explicite (vezi în semnătură ce-i trebuie), ușor de testat fără Spring (`new Alerts(mock)`).
- De la Spring 4.3+, dacă există un *singur* constructor public, `@Autowired` e *implicit* — nu trebuie scris.

4.2 Field injection (anti-pattern ❌)

```
@Service
public class Alerts {
    @Autowired
    private NotificationSender sender;  // nu poți să-l faci final
}
```

- Funcționează, dar: nu poți instanția clasa fără Spring (testabilitate redusă), dependențele sunt ascunse, câmpul nu poate fi `final`.
- O să vezi cod existent care arată așa — *nu copia stilul*.

4.3 Setter injection (rar)

```
@Autowired
public void setSender(NotificationSender sender) { this.sender = sender; }
```

Folosit ocazional pentru dependențe opționale. În practică modernă: rar.

4.4 `@Autowired` explicit pe parametru

Pe parametrii de constructor sau metode, `@Autowired` e implicit. Îl scrii doar când vrei să-l combini cu altă adnotare (de ex. `@Qualifier`):

```
public Alerts(@Qualifier("email") NotificationSender sender) { ... }
```

5. Mai multe bean-uri de același tip

Când Spring găsește 2+ bean-uri care implementează același tip, nu știe pe care să-l aleagă → `NoUniqueBeanDefinitionException` la pornire.

5.1 `@Qualifier` — alegi explicit

```
@Component("email") class EmailSender implements NotificationSender { ... }
@Component("sms")   class SmsSender   implements NotificationSender { ... }

@Component
class Alerts {
    Alerts(@Qualifier("email") NotificationSender sender) { ... }
}
```

5.2 `@Primary` — alege default

```
@Component @Primary class StripeGateway implements PaymentGateway { ... }
@Component          class PaypalGateway implements PaymentGateway { ... }

@Component
class Checkout {
    // primește Stripe (e @Primary)
    Checkout(PaymentGateway gateway) { ... }
}
```

- `@Qualifier` are prioritate peste `@Primary` dacă-l scrii explicit.
- Folosește `@Primary` când 90% dintre consumatori vor implementarea "obișnuită" și doar câțiva cer alternativa.

6. Configurare externă

6.1 `@Value` — valori scalare

```
@Component
class AppProperties {
    private final String greeting;

    AppProperties(@Value("${app.greeting}") String greeting) {
        this.greeting = greeting;
    }
}
```

În `application.yaml`:

```
app:
  greeting: "Hello"
```

- Sintaxă: `${cheie}` citește din yaml/properties/env.
- Default dacă lipsește: `@Value("${cheie:fallback}")`.
- Funcționează și pentru `int`, `boolean`, `long` etc. — Spring face conversia.

6.2 `@ConfigurationProperties` (vine în L02)

Pentru un grup de proprietăți, mai curat decât multiple `@Value`:

```
@ConfigurationProperties(prefix = "app")
public record AppConfig(String greeting, int retries) {}
```

7. Bean lifecycle & scope

7.1 `@PostConstruct` — după DI, înainte să fie folosit

```
@Component
class CacheWarmer {
    private List<String> cache = new ArrayList<>();

    @PostConstruct
    void init() {
        cache.add("warm-up-data");
    }
}
```

Util pentru: pre-încărcare cache, validări de config la pornire, conexiuni externe.

7.2 `@PreDestroy` — înainte ca containerul să închidă

```
@PreDestroy
void shutdown() { /* eliberează resurse */ }
```

7.3 `@Scope` — câte instanțe?

scope	semnificație
`singleton` (default)	**o singură instanță** pe toată durata vieții containerului
`prototype`	instanță nouă la *fiecare cerere* (`@Autowired` sau `getBean`)
`request` / `session`	pe request HTTP / sesiune (doar în web apps)

```
@Component
@Scope("prototype")
class CartSession { ... }
```

Atenție: dacă injectezi un bean `prototype` într-un `singleton`, primești tot o singură instanță (cea care exista când singleton-ul s-a construit). Soluții: `Provider<T>`, `ObjectFactory<T>`, sau scoped proxy — depășește L01.

8. Optimizare

`@Lazy` — amână crearea bean-ului

Default: Spring construiește toate bean-urile la pornire. Cu `@Lazy`, le creează *prima dată când e cerut*:

```
@Component
@Lazy
class HeavyService { ... }
```

Util pentru bean-uri scumpe care de obicei nu sunt necesare în run-time normal.

9. Activare condițională

`@Profile` — bean diferit per environment

```
@Configuration
@Profile("dev")
class DevDataConfig { @Bean DataSource ds() { /* H2 */ } }

@Configuration
@Profile("prod")
class ProdDataConfig { @Bean DataSource ds() { /* MySQL */ } }
```

Activezi profilul cu `--spring.profiles.active=dev` sau în `application.yaml`.

`@ConditionalOnProperty`, `@ConditionalOnClass` etc.

Auto-configurarea Spring Boot le folosește intern. Le vei vedea când citești cod open-source. **Nu** sunt necesare în L01.

10. Ordine

`@DependsOn` — forțează ordinea de creare

```
@Component
@DependsOn("databaseInitializer")
class UserService { ... }
```

Rar necesar — Spring rezolvă ordinea automat din DI. Când ai nevoie: când o dependență e *implicită* (nu apare în constructor) dar trebuie să existe înainte.

11. Adnotări la teste

`@SpringBootTest`

Pornește un `ApplicationContext` complet pentru test. Folosit în L01 ca să verificăm wiring-ul de bean-uri.

```
@SpringBootTest
class Ex01GreetingServiceTest {
    @Autowired GreetingService greetingService;
    // ...
}
```

Pentru lecții ulterioare: există `@WebMvcTest` (doar layer-ul web), `@DataJpaTest` (doar layer-ul JPA), `@MockBean` etc. — vin la momentul lor.

Hartă: ce înveți pe rând

L01 (acum)	L02+
Container, DI, bean-uri, scope, lifecycle	REST controllers, validation, JPA, security, testing avansat

Cheat sheet — adnotări de bază

adnotare	unde	rol
`@SpringBootApplication`	clasa cu main()	bootstrap + scan + auto-config
`@Configuration`	clasă	sursă de bean-uri prin `@Bean`
`@Bean`	metodă în `@Configuration`	publică obiectul returnat ca bean
`@Component`	clasă	bean generic descoperit la scan
`@Service`	clasă	bean cu rol de business logic
`@Repository`	clasă	bean cu rol de data access
`@Controller` / `@RestController`	clasă	bean cu rol de endpoint HTTP (L02+)
`@Autowired`	constructor / câmp / setter	cere injecție
`@Qualifier("nume")`	parametru	alege bean specific
`@Primary`	clasă / `@Bean`	bean default când există mai multe
`@Value("${cheie}")`	parametru / câmp	injectează valoare din config
`@PostConstruct`	metodă fără args	rulează după DI
`@PreDestroy`	metodă fără args	rulează înainte de shutdown
`@Scope("prototype")`	clasă / `@Bean`	una nouă la fiecare cerere
`@Lazy`	clasă / `@Bean` / parametru	creare amânată

adnotare	unde	rol
`@Profile("dev")`	clasă / `@Bean`	activ doar pe profilul respectiv
`@DependsOn("alt-bean")`	clasă	forțează ordinea de construcție